

AZURE DATA ENGINEERING PROJECT

End-to-End Retail Data Engineering Pipeline

Azure SQL + Azure Data Factory + ADLS Gen2 + Databricks + Power BI

Project outcome

A beginner-friendly implementation of a retail analytics pipeline that ingests structured SQL data and customer JSON data, stores raw data in a Bronze layer, transforms it in Databricks, publishes analytics-ready data, and presents the results through a Power BI Retail Sales Dashboard.

Project Report

Prepared as a step-by-step implementation guide and documentation report

Table of Contents

1. Project Overview
2. Business Requirement
3. Solution Architecture
4. Tools and Technologies
5. Step 1 – Create Azure Resources
6. Step 2 – Create ADLS Gen2 Storage Structure
7. Step 3 – Create Azure SQL Database and Tables
8. Step 4 – Validate Source Data
9. Step 5 – Create the ADLS Bronze Layer
10. Step 6 – Build the Azure Data Factory Pipeline
11. Step 7 – Ingest Customer JSON Data
12. Step 8 – Configure Databricks Access
13. Step 9 – Build the Databricks Transformation Layer
14. Step 10 – Create Gold/Analytics Data
15. Step 11 – Create Databricks SQL Warehouse
16. Step 12 – Connect Power BI to Databricks
17. Step 13 – Build the Power BI Dashboard
18. Data Validation and Expected Results
19. Final Architecture and Project Flow
20. Beginner Troubleshooting Guide
21. Interview Explanation
22. Conclusion

1. Project Overview

The project demonstrates how a retail organization can move data from operational sources into a cloud data platform and then expose business insights through Power BI. The pipeline combines structured Azure SQL data with customer JSON data, uses Azure Data Factory for ingestion, Azure Data Lake Storage Gen2 for storage, Databricks for transformation, and Power BI for visualization.

What a beginner should understand

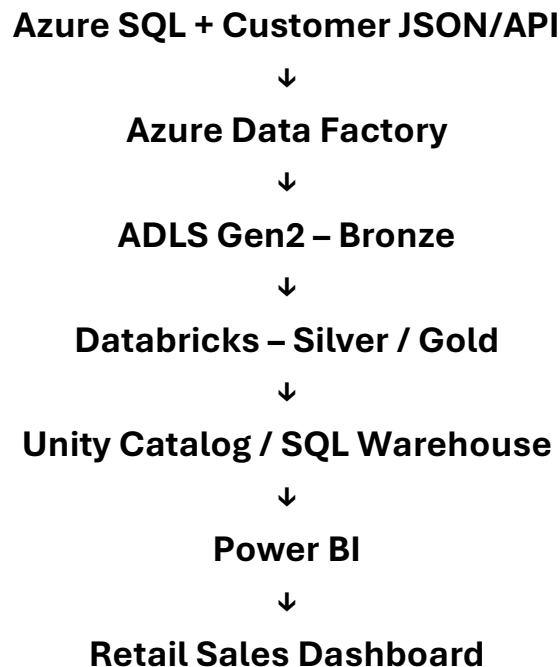
Think of the project as a factory: Azure SQL/API are the suppliers, ADF is the transport system, ADLS is the warehouse, Databricks is the cleaning/processing area, the Gold layer is the finished product, and Power BI is the showroom.

2. Business Requirement

- Collect product, store, and transaction data from a relational source.
- Collect customer information from JSON/API-style data.
- Land raw data in an Azure Data Lake Bronze layer.
- Clean and transform the data using Databricks.
- Create analytics-ready datasets for reporting.
- Connect Power BI and build a Retail Sales Dashboard.

3. Solution Architecture

The end-to-end flow used in this project is:



4. Tools and Technologies

Technology	Purpose	Beginner meaning
Azure SQL Database	Source tables	Stores structured retail data
Azure Data Factory	Data ingestion/orchestration	Moves data from source to lake
ADLS Gen2	Cloud data lake	Stores Bronze/Silver/Gold data
Azure Databricks	Data engineering	Cleans, transforms and prepares data
Unity Catalog	Governance/catalog	Organizes and controls tables
Databricks SQL Warehouse	SQL/BI serving	Provides a SQL endpoint for Power BI
Power BI	Reporting	Turns data into charts and KPIs

5. Step 1 – Create Azure Resources

Start in the Azure Portal. Create the resources needed for the project and keep them in one resource group where practical.

1. Open the Azure Portal and sign in.
2. Create or select a resource group for the project.
3. Create an Azure Storage Account with hierarchical namespace enabled so it can be used as ADLS Gen2.
4. Create an Azure SQL Database.
5. Create an Azure Data Factory instance.
6. Create or open an Azure Databricks workspace.
7. Keep the resource names organized and record them in the project documentation.

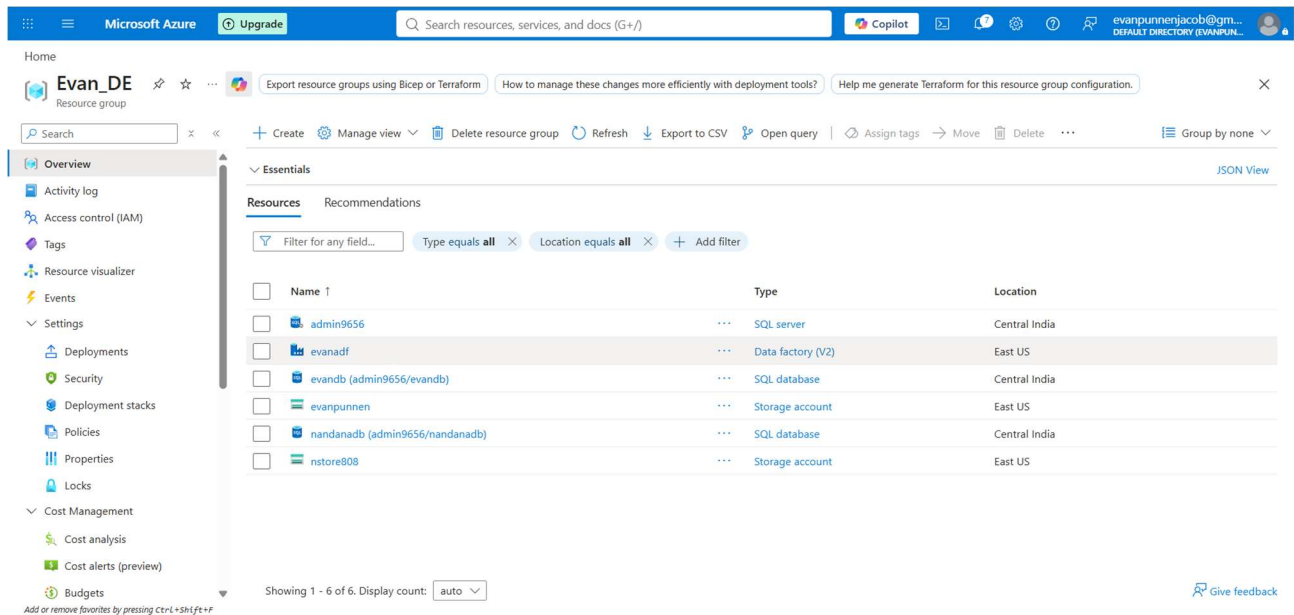


Figure 1. Azure resources used for the project.

6. Step 2 – Create ADLS Gen2 Storage Structure

The data lake is organized into layers. The Bronze layer keeps the ingested/raw representation, while Silver and Gold are used for processed data.

8. Open the Storage Account.
9. Open Containers.
10. Create the Bronze container.
11. Inside Bronze, create folders for customers, products, stores, and transactions.
12. Later create Silver and Gold containers for transformed and analytics-ready data.

Microsoft Azure | Upgrade | Search resources, services, and docs (G+/) | Copilot | evanpunnenjacob@gmail... | evanpunnenjacob@gmail... | evanpunnenjacob@gmail... | evanpunnenjacob@gmail...

Home > evanpunnen

evanpunnen | Containers | Storage account

Search

Overview
Activity log
Tags
Diagnose and solve problems
Access Control (IAM)
Data migration
Events
Storage browser
Partner solutions
Resource visualizer
Data storage
Containers
Classic file shares
Queues
Tables
Security + networking
Networking

Add container | Upload | Refresh | Delete | Change access level | Restore containers | Edit columns

Search containers by prefix | Only show active containers

Showing all 4 items

☐	Name	Last modified	Anonymous access level	Lease state	
☐	 Slogs	9/24/2026, 12:21:18 PM	Private	Available	...
☐	 bronze	9/24/2026, 12:31:06 PM	Private	Available	...
☐	 gold	9/24/2026, 12:31:54 PM	Private	Available	...
☐	 silver	9/24/2026, 12:31:14 PM	Private	Available	...

Add or remove favorites by pressing Ctrl+Shift+F

Figure 2. ADLS Gen2 containers used by the retail pipeline.

7. Step 3 – Create Azure SQL Database and Tables

Create the relational source tables. The project uses Products, Stores and Transactions as the main structured retail sources.

7.1 Products

Typical columns: product_id, product_name, category and price.

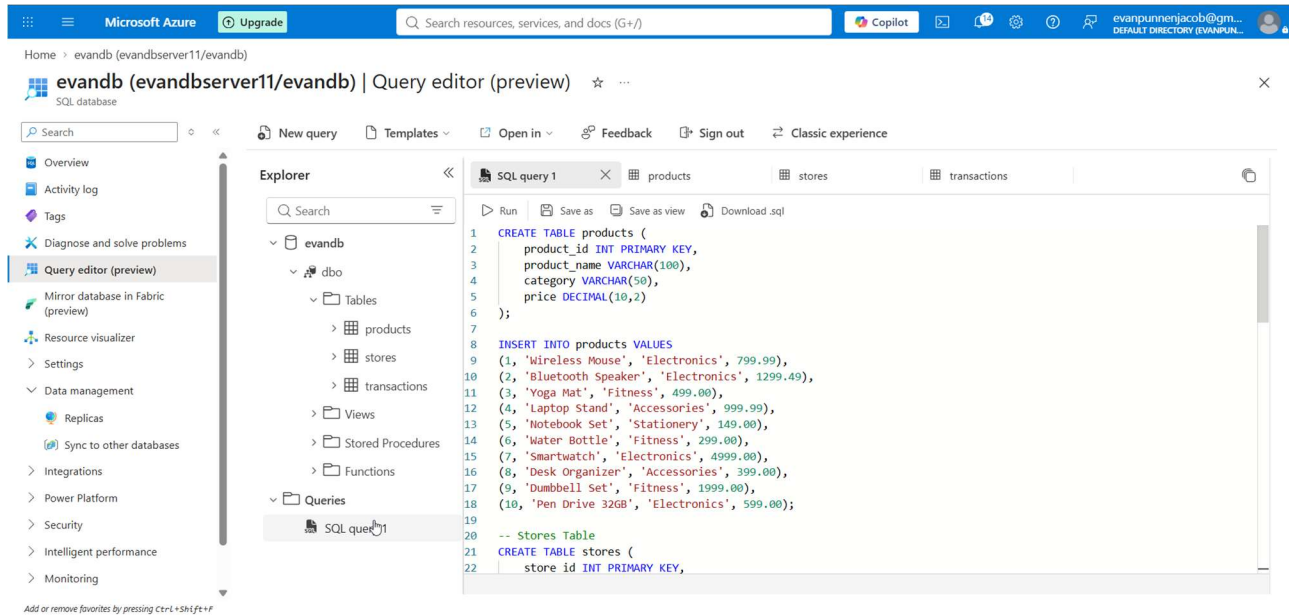


Figure 3. Azure SQL query editor showing the Products source table.

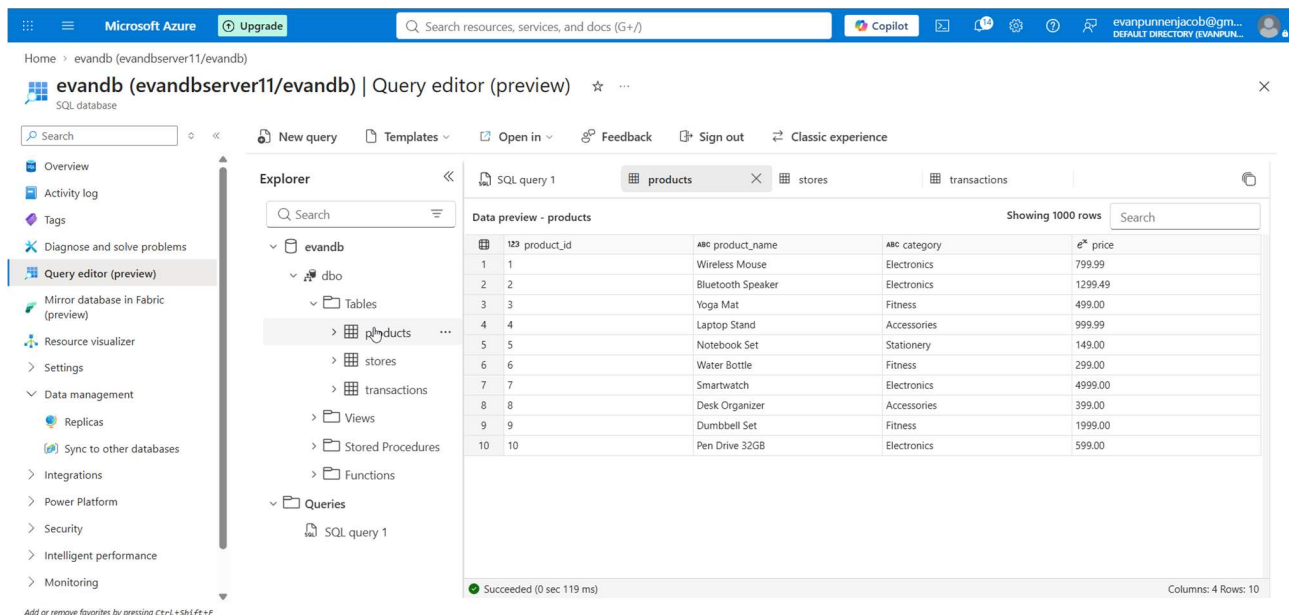


Figure 4. Products table data preview.

7.2 Stores

Typical columns: store_id, store_name and location.

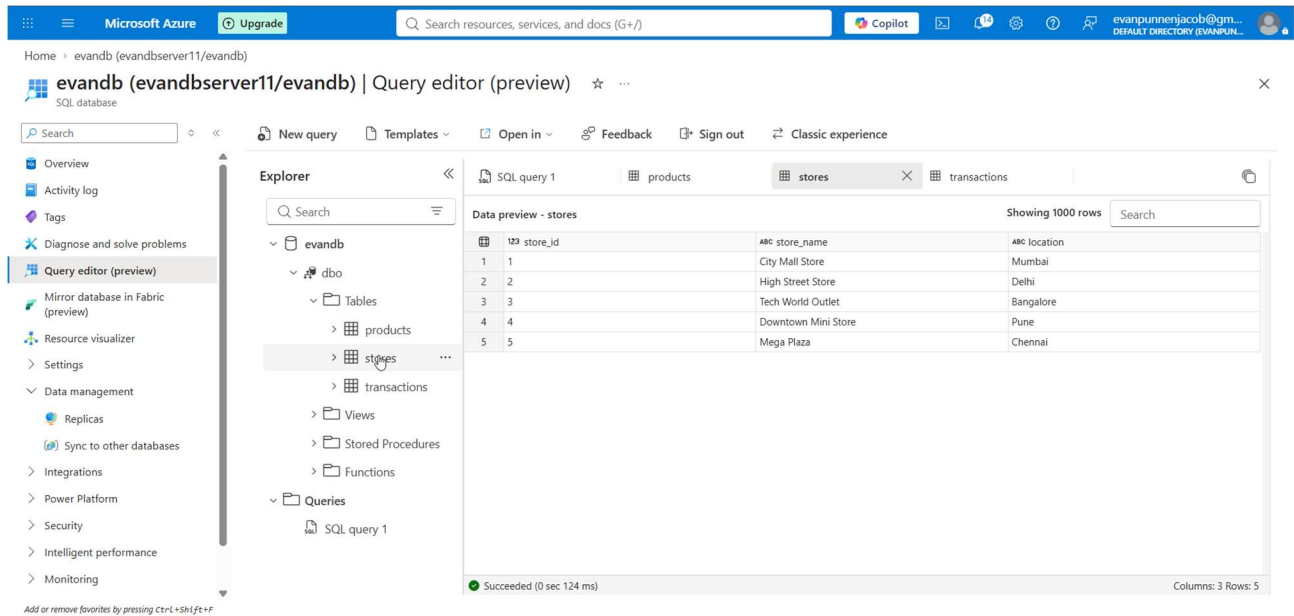


Figure 5. Stores table data preview.

7.3 Transactions

Typical columns: transaction_id, customer_id, product_id, store_id, quantity and transaction_date.

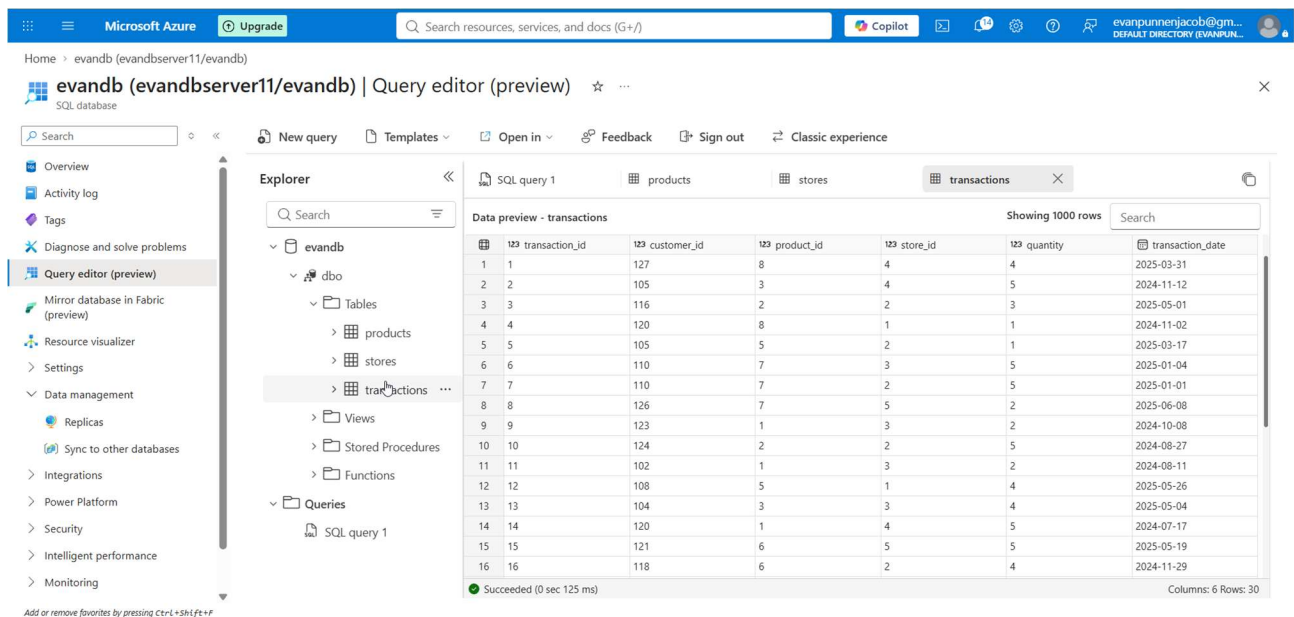


Figure 6. Transactions table data preview.

8. Step 4 – Validate Source Data

Before building pipelines, verify that the source tables contain the expected records and that primary/foreign-key relationships make sense.

- Products: 10 records.
- Stores: 5 records.
- Transactions: 30 records.
- Customer source: 50 records.
- Check that product_id and store_id values used by transactions exist in their parent tables.

9. Step 5 – Create the ADLS Bronze Layer

Bronze is the landing zone. The goal is to copy source data into the lake with minimal business transformation.

13. Open the Bronze container.
14. Create or confirm the customers, products, stores and transactions folders.
15. Use the folder names consistently in ADF sink datasets.

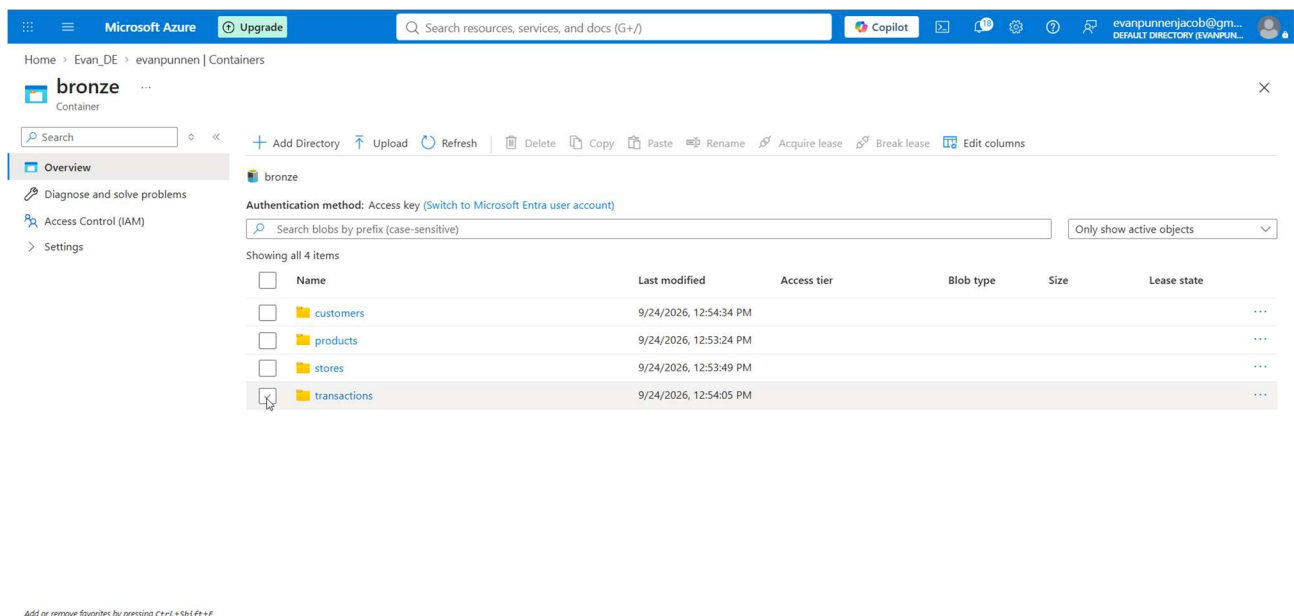


Figure 7. Bronze layer folders after source ingestion.

10. Step 6 – Build the Azure Data Factory Pipeline

ADF is responsible for moving source data into the Bronze layer.

16. Open Azure Data Factory and launch Author & Monitor.
17. Create a linked service to Azure SQL.
18. Create source datasets for Products, Stores and Transactions.
19. Create ADLS sink datasets pointing to the Bronze folders.
20. Create Copy activities for Products, Stores and Transactions.
21. Connect the activities in the pipeline.
22. Validate the pipeline.
23. Publish the changes.
24. Trigger the pipeline and check Monitor for successful runs.

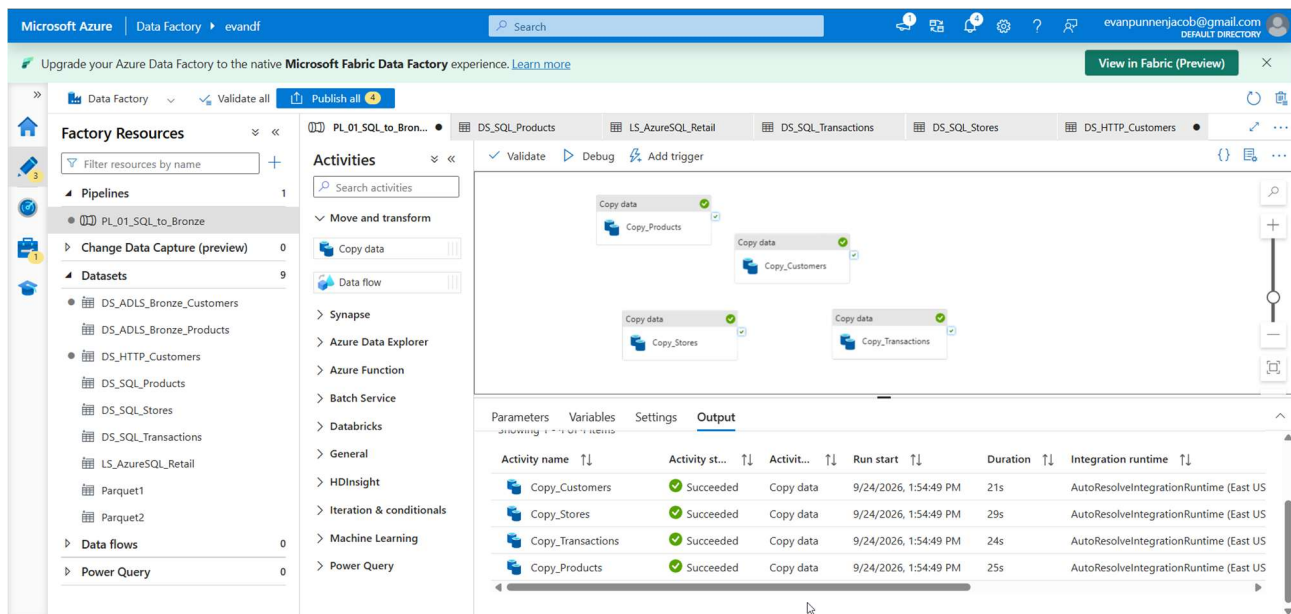


Figure 8. ADF pipeline showing the ingestion activities and successful execution.

11. Step 7 – Ingest Customer JSON Data

Customer information is provided as JSON. It is stored in the Bronze customers folder so that Databricks can process it together with the other datasets.

25. Obtain the customer JSON source.
26. Create the appropriate ADF/ADLS ingestion step or upload the source into the Bronze customers folder, depending on the chosen implementation.
27. Confirm that the JSON contains customer_id, name, email, phone, city and registration information.
28. Open the JSON in ADLS and verify that it is readable.

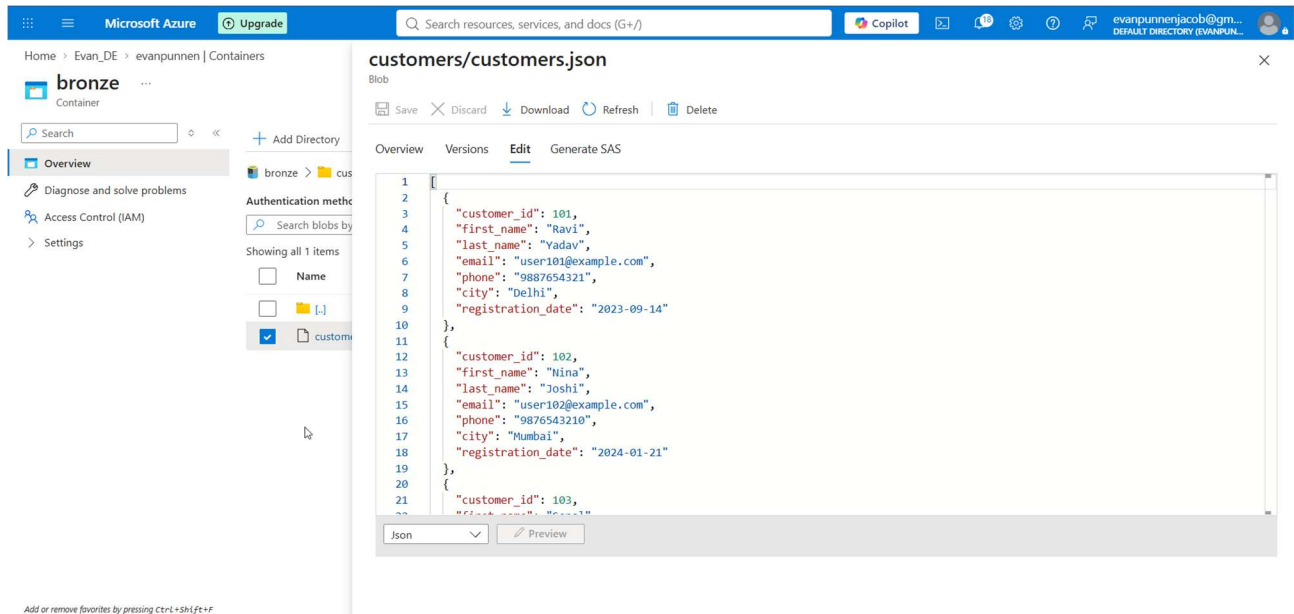


Figure 9. Customer JSON stored in the Bronze layer.

12. Step 8 – Configure Databricks Access

Databricks needs permission to access the ADLS data. In the implemented setup, storage access is configured through Unity Catalog and the required Azure identity/credential configuration.

29. Open the Azure Databricks workspace.
30. Configure the required storage credential / managed identity access.
31. Grant the identity the required Storage Blob Data access on the storage account.
32. Create or verify the external location for the relevant ADLS path.
33. Use Unity Catalog to organize the project data.

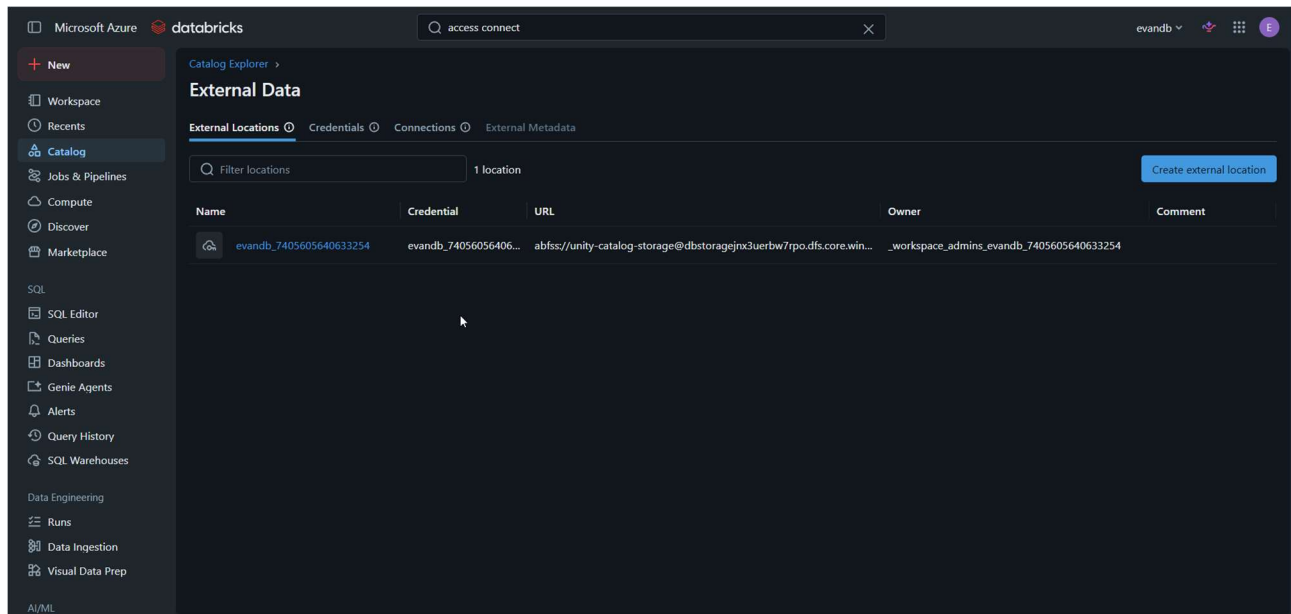


Figure 10. Databricks external-data/storage configuration.

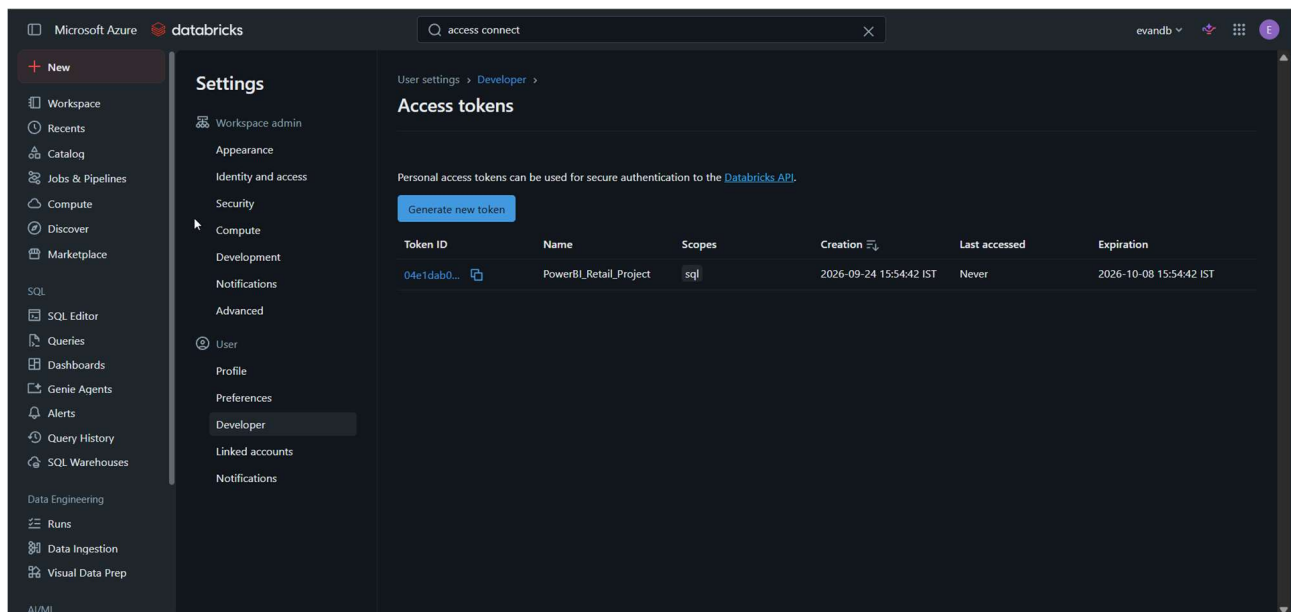


Figure 11. Databricks access-token settings used during development/authentication.

Security note

Never put an access token, password, connection secret, or client secret into a report, GitHub repository, screenshot, or chat message. Tokens shown in development should be treated as sensitive credentials.

13. Step 9 – Build the Databricks Transformation Layer

Databricks reads the Bronze data, applies data-quality rules, and writes processed datasets into the Silver/Gold layers.

34. Create a Databricks notebook for the retail pipeline.
35. Read Products, Stores and Transactions from Bronze.
36. Read Customer JSON with the appropriate JSON options when required.
37. Remove duplicates.
38. Trim text fields.
39. Cast numeric/date columns to the correct data types.
40. Check nulls and invalid keys.
41. Validate referential integrity between transactions and dimension data.
42. Write the cleaned datasets to the Silver layer.

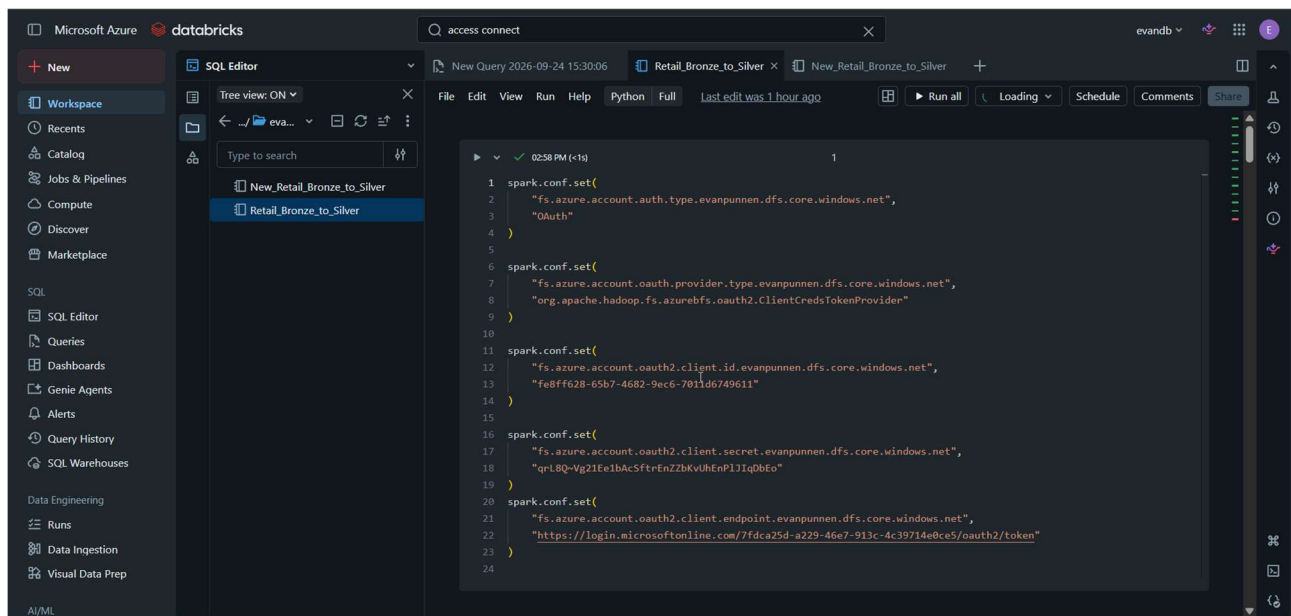


Figure 12. Databricks notebook transformation logic.

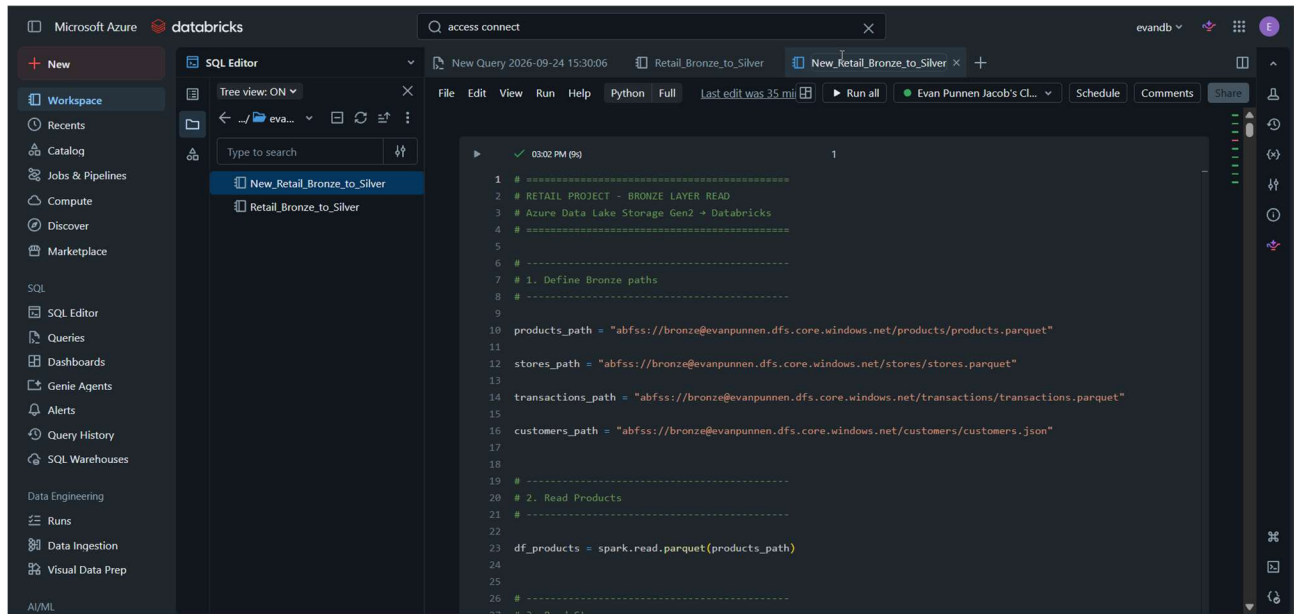


Figure 13. Databricks processing code for the project.

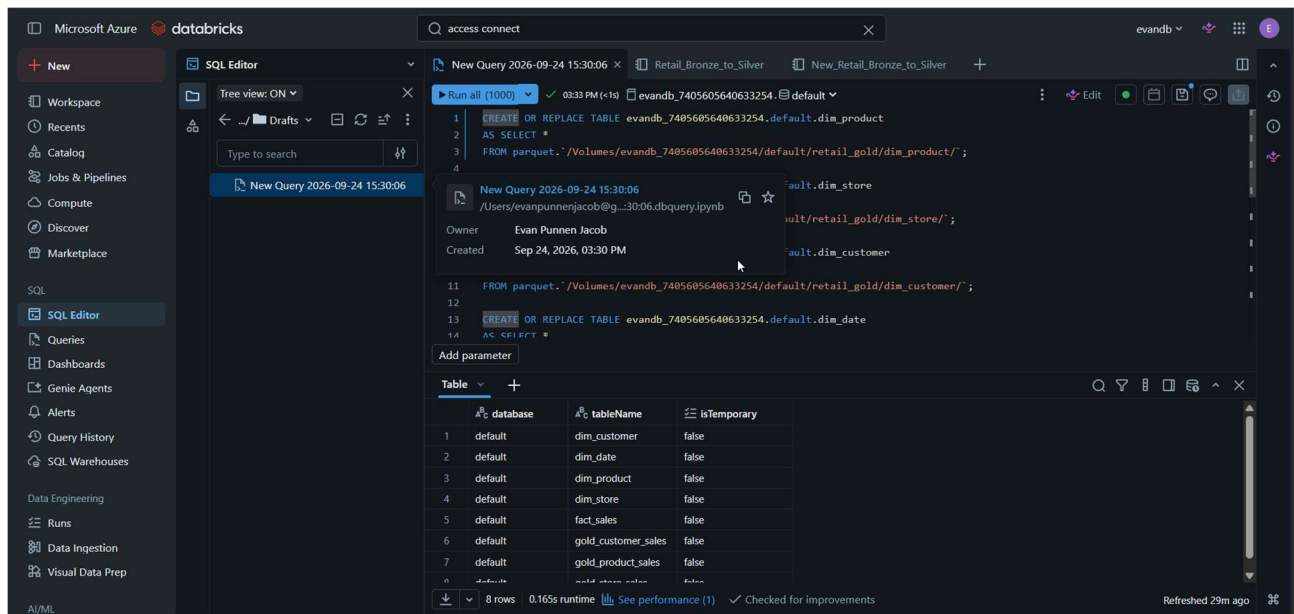


Figure 14. Databricks table/output after transformation.

14. Step 10 – Create Gold/Analytics Data

The Gold layer is designed for business reporting. A star-schema approach is used with dimensions and a sales fact table.

- dim_product – product attributes.
- dim_store – store attributes.
- dim_customer – customer attributes.
- dim_date – date attributes.
- fact_sales – transaction-level sales facts including quantity, unit price and total amount.

Additional aggregated reporting tables can be created for product, store and customer sales summaries when required.

15. Step 11 – Create Databricks SQL Warehouse

A SQL Warehouse provides the SQL serving layer used by Power BI.

- 43. Open Databricks SQL / SQL Warehouses.
- 44. Create or start the SQL Warehouse.
- 45. Confirm that the warehouse is running.
- 46. Record the Server Hostname and HTTP Path for the Power BI connection.

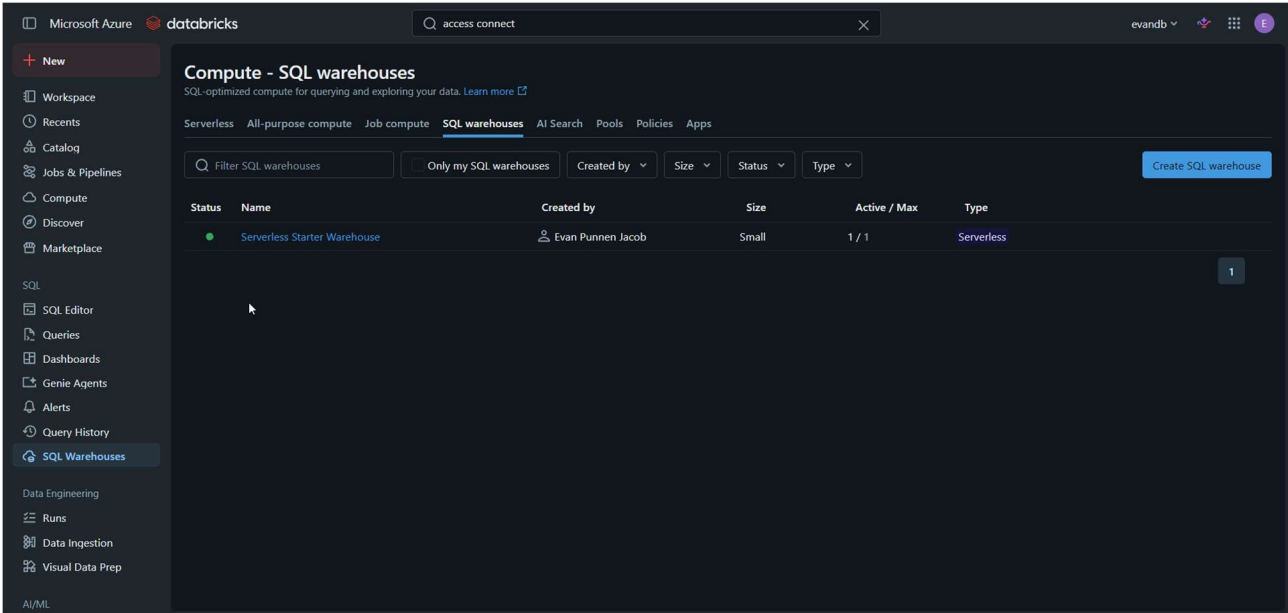


Figure 15. Databricks SQL Warehouse used as the BI serving endpoint.

16. Step 12 – Connect Power BI to Databricks

Open Power BI Desktop and connect using the Azure Databricks connector. Authenticate with the account/credential configured for the workspace.

47. Open Power BI Desktop.
48. Select Get Data → Azure Databricks.
49. Enter the Databricks Server Hostname.
50. Enter the SQL Warehouse HTTP Path.
51. Choose the supported authentication method for the account/environment.
52. After authentication, select the project catalog and schema.
53. Select the required reporting tables.
54. Load the data or open Power Query when transformations are needed.

17. Step 13 – Build the Power BI Dashboard

The reference dashboard contains a store-sales bar chart, a sales-over-time line chart, a category sales donut chart, and KPI cards.

55. Create a report page named Retail Sales Dashboard.
56. Add a card for Total Sales / Revenue.
57. Add a card for Total Transactions. Use Count of transaction_id, not Sum of transaction_id.
58. Add a bar chart using store_name on the axis and sales/revenue as the value.
59. Add a line chart using transaction_date on the axis and sales/revenue as the value.
60. Add a donut chart using category and sales/revenue.
61. Add a text box with the title Retail Sales Dashboard.
62. Format the visuals consistently and align them on the page.
63. Save the PBIX file.

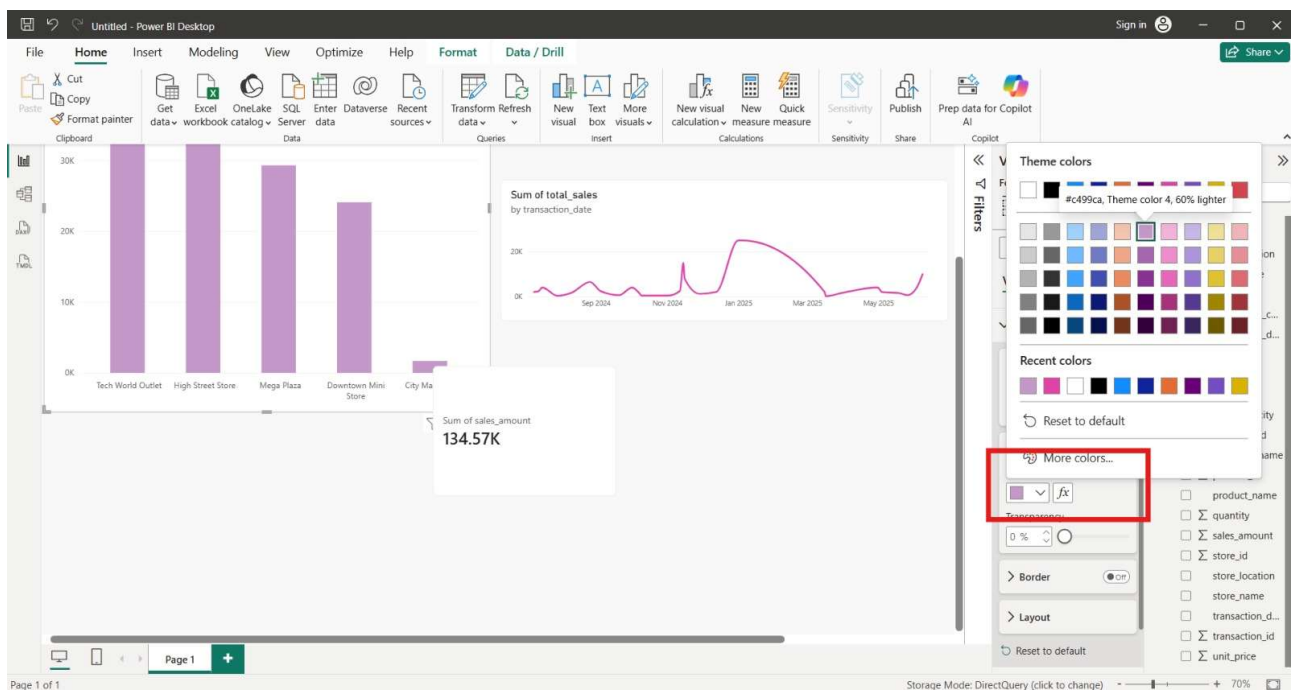


Figure 16. Power BI visual formatting.

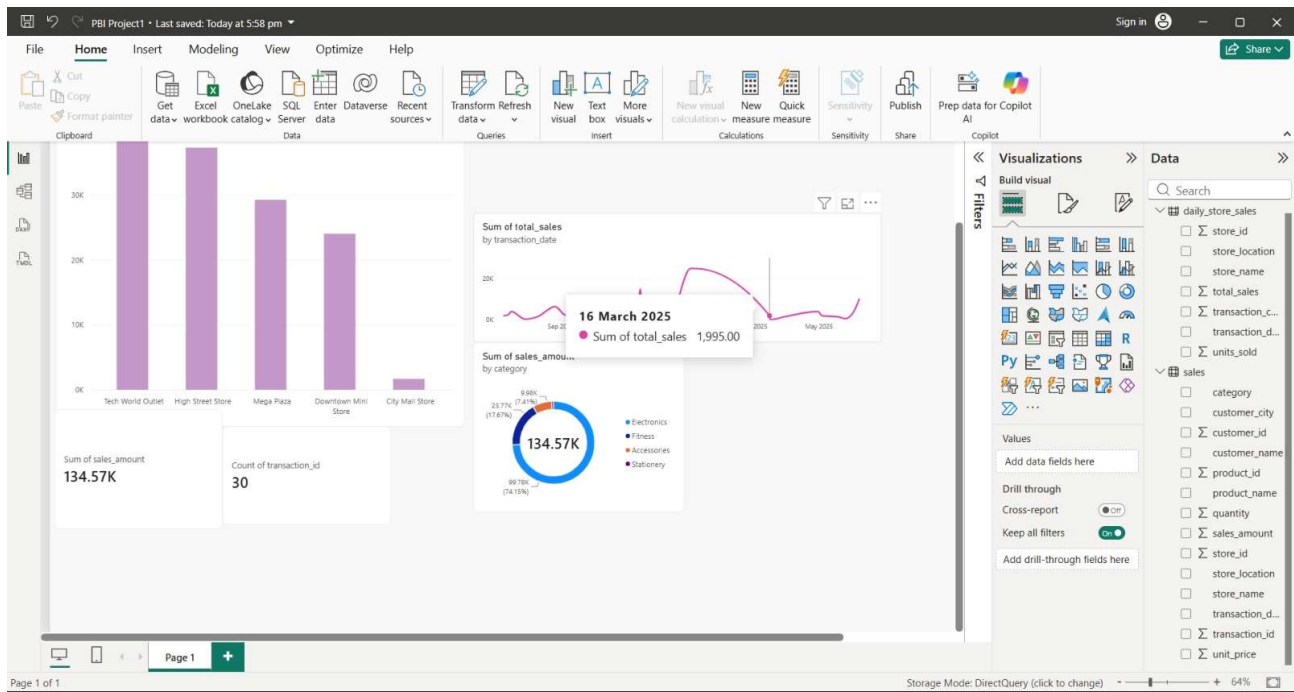


Figure 17. Power BI dashboard visuals and sales trend.

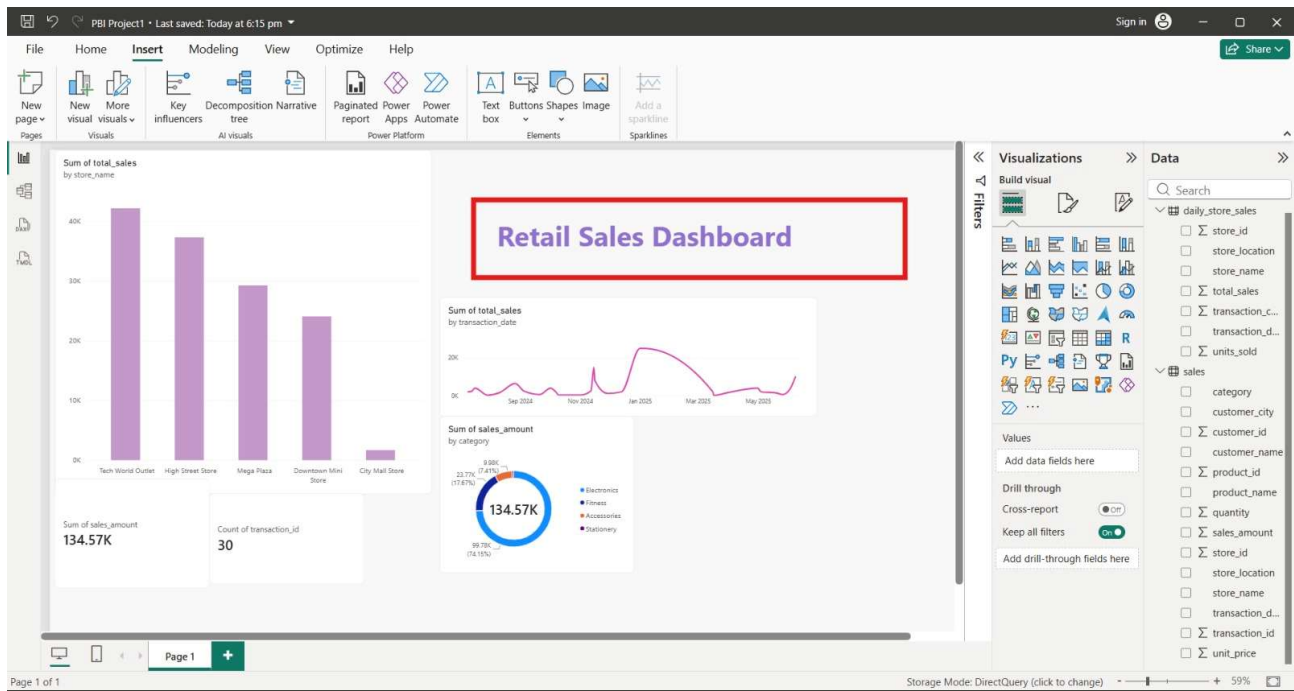


Figure 18. Retail Sales Dashboard title and layout.

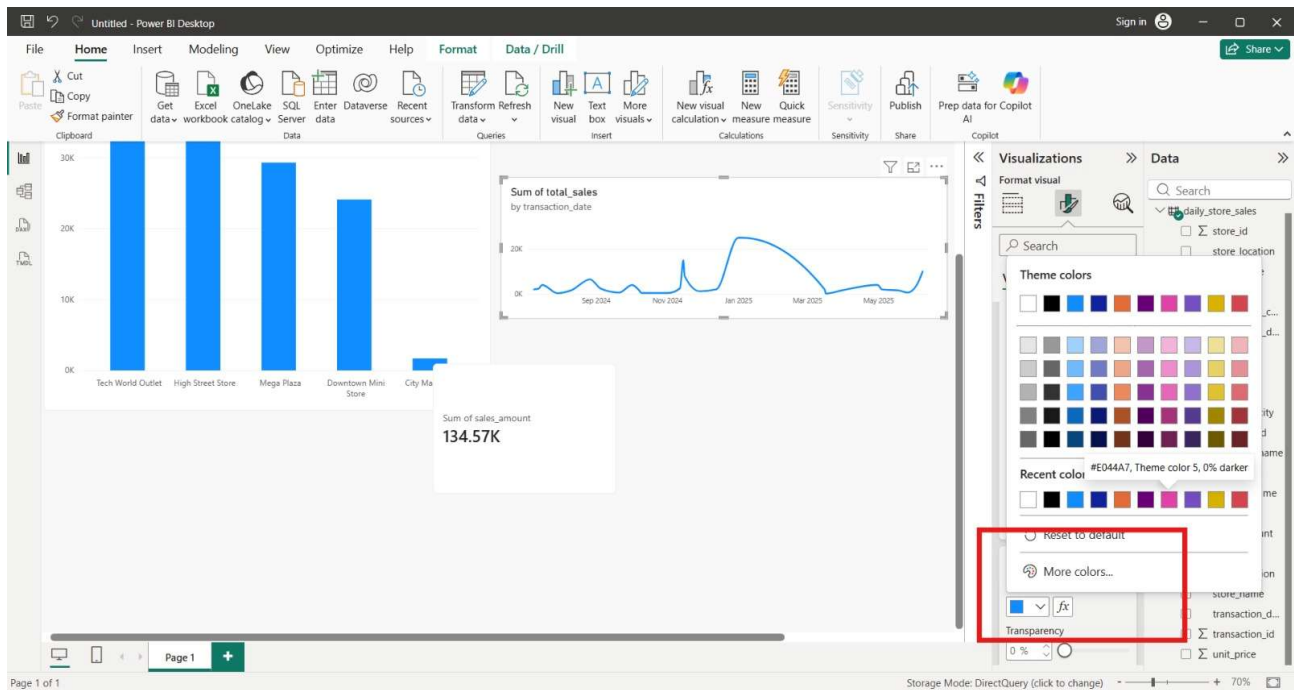


Figure 19. Power BI visual color formatting.

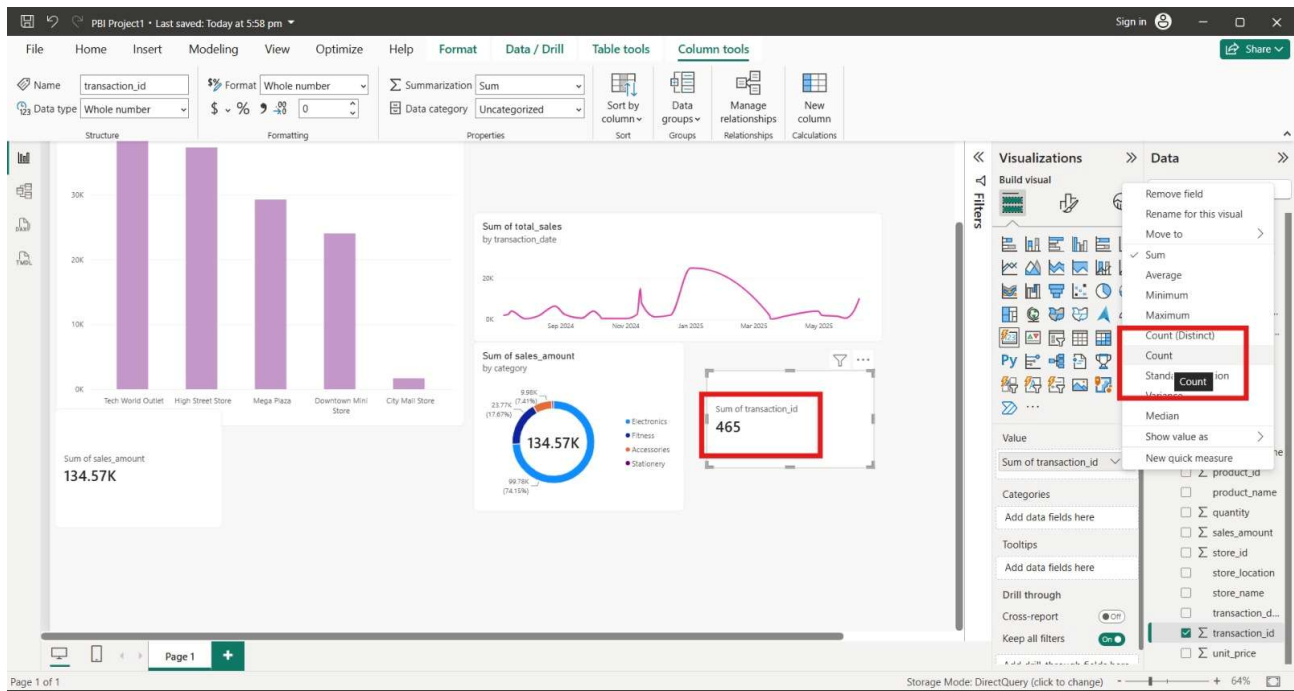


Figure 20. Correct transaction-count configuration using Count.

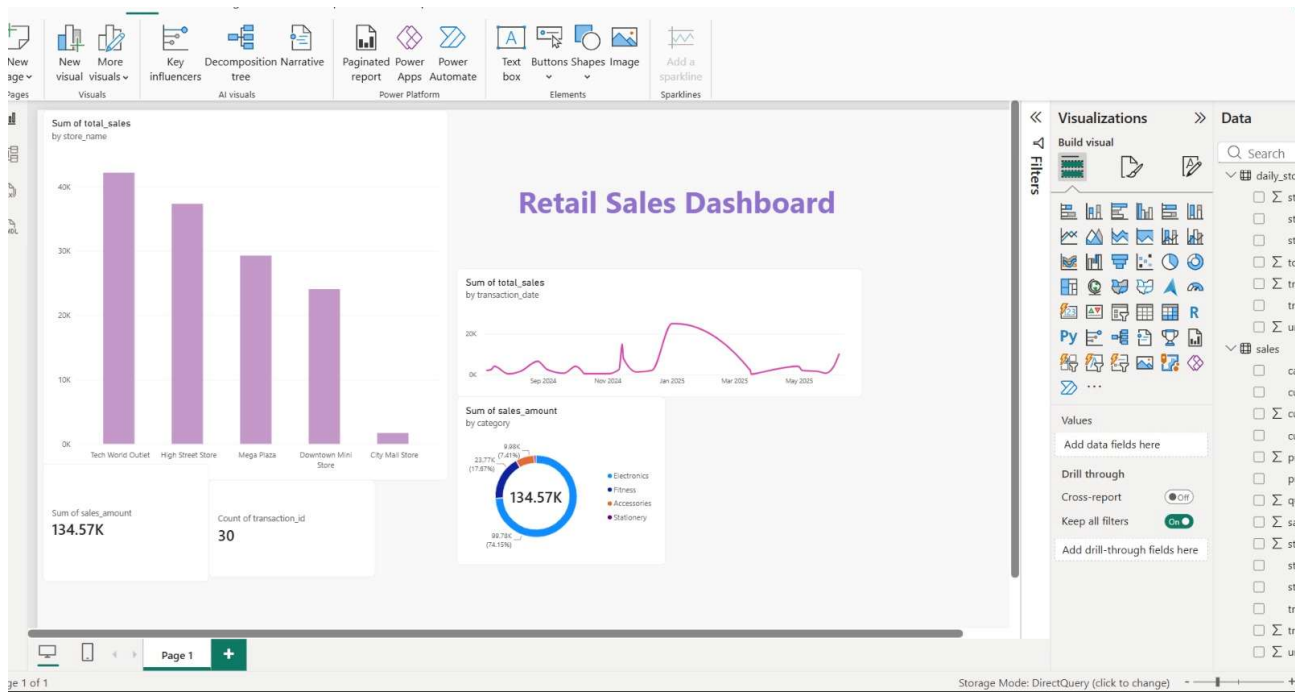


Figure 21. Final Retail Sales Dashboard.

18. Data Validation and Expected Results

The following checks should be used to confirm that the pipeline has not changed the business meaning of the data.

Check	Expected	Why it matters
Products	10	Confirms source-to-lake ingestion
Stores	5	Confirms store master data
Transactions	30	Confirms transaction ingestion
Customers	50	Confirms customer JSON ingestion
Total transactions KPI	30	Must count transaction rows, not sum IDs
Total sales	134.57K in the reference dashboard	Confirms the dashboard aggregation shown in the supplied screenshots

Important Power BI check

If transaction_id is used directly in a Card and summarized as Sum, Power BI will add the ID values. For a transaction count, change the aggregation to Count or create a dedicated measure such as `COUNT(fact_sales[transaction_id])`.

19. Final Architecture and Project Flow

- 1. Source systems – Azure SQL and customer JSON.
- 2. Ingestion – Azure Data Factory.
- 3. Storage – ADLS Gen2 Bronze.
- 4. Processing – Databricks Silver.
- 5. Business-ready layer – Databricks Gold.
- 6. Serving – Unity Catalog / SQL Warehouse.
- 7. Reporting – Power BI.

20. Beginner Troubleshooting Guide

Problem	What to check
ADF copy fails	Check the Azure SQL linked service, credentials, dataset/table name, sink path, and permissions.
Customer JSON appears as corrupt	For multi-line JSON, use the appropriate multiLine JSON read option in Databricks and verify the file

	itself is valid JSON.
Databricks cannot read ADLS	Check the storage credential, managed identity, Azure RBAC permissions, external location and path.
Unity Catalog external location error	Check whether the path already belongs to another external location or has dependent resources. Avoid creating overlapping locations.
Power BI authentication fails	Make sure the selected identity belongs to the tenant that can access the Databricks workspace. Try the authentication method supported by the environment.
Power BI shows Sum of transaction_id	Change the field aggregation to Count; transaction IDs are identifiers, not a measure to sum.
Dashboard values look wrong	Check relationships, duplicate fact rows, date types, and the aggregation used by each visual.

21. Interview Explanation

A beginner-friendly way to explain the project in an interview:

“I built an end-to-end Azure retail data engineering pipeline. The structured retail data comes from Azure SQL and customer data is provided as JSON. Azure Data Factory is used for ingestion and orchestration, and the raw data is stored in ADLS Gen2 Bronze. Databricks cleans and transforms the data into Silver and Gold layers, with a star-schema reporting model containing customer, product, store and date dimensions and a sales fact table. Databricks SQL Warehouse provides the serving endpoint, and Power BI connects to it to build the Retail Sales Dashboard. I also performed validation checks for row counts, nulls, duplicates and referential integrity.”

22. Conclusion

This project demonstrates the complete path from operational data to business reporting on Azure. The most important learning is not a single tool but how the tools work together: ADF moves data, ADLS stores it, Databricks transforms it, Unity Catalog organizes access, SQL Warehouse serves it, and Power BI communicates the results.

Final checklist

- Azure resources created ✓
- ADLS Bronze structure ✓
- Azure SQL source tables ✓
- ADF ingestion pipeline ✓
- Customer JSON ingestion ✓
- Databricks transformation ✓
- Silver/Gold analytics layer ✓
- SQL Warehouse ✓

